Phase 0 · Production document

Technical Design Document

How the game is built: project setup, module design, events, data formats, Blender conventions, saves, AI, rendering, audio, performance and deployment.

Stack	TypeScript 6 · Babylon.js 9.31 · Havok 1.3 · Vite 8
Extends	Development Plan, section 3
Version	1.1 · October 2026
Author	Rithy Bondeth

Contents

01 Principles

02 Project Setup and Babylon 9 Pitfalls

03 Modules

04 Boot and Game Loop

05 Event Catalogue

06 Data Formats

07 Blender and glTF Conventions

08 Asset Pipeline

09 Scripted Sequences

10 Perception and AI

11 Rendering

12 Audio

13 Saves and Save Codes

14 Performance and Memory

15 Testing, Standards and Deployment

01 Principles

Principle	In practice
Systems are generic; episodes are data	No episode-specific code in `systems/`. Episodes provide data files, Blender scenes and a small number of sequence scripts.
Loose coupling	Systems communicate through a typed event bus and the flag store, never by importing each other.
Blender is the level editor	Placement of triggers, items, patrols and zones lives in Blender, exported as glTF extras.
Use the engine	Physics, audio, picking, GUI and post-processing come from Babylon.js; we write glue and game logic, not engine features.
Testable logic	Game rules (inventory, flags, saves, AI decisions) are plain TypeScript that can be unit-tested without a renderer.

API names below match Babylon.js 9. Check the current documentation when implementing, since minor APIs change between releases.

02 Project Setup and Babylon 9 Pitfalls

2.1 Dependencies (pinned exact versions)

```
"dependencies": {
  "@babylonjs/core":    "9.31.0",
  "@babylonjs/loaders": "9.31.0",
  "@babylonjs/gui":     "9.31.0",
  "@babylonjs/havok":   "1.3.14",
  "idb-keyval":         "(latest at setup)"
},
"devDependencies": {
  "typescript": "6.0.3",            // TypeScript 7 not yet supported by typescript-eslint
  "vite":       "8.3.4",
  "vitest":     "5.0.5",
  "eslint":     "10.13.0",  "typescript-eslint": "8.72.0",
  "prettier":   "3.9.10",
  "@gltf-transform/cli": "4.5.1",
  "@playwright/test":   "(add with the smoke test)"
}
```

2.2 Scripts

Command	Does
`npm run dev`	Vite dev server with hot reload
`npm run build`	Type-check and production build to `dist/`
`npm run test`	Vitest unit tests
`npm run lint`	ESLint with type-aware TypeScript rules
`npm run check`	Everything CI runs; GitHub Actions runs it on every push (`.github/workflows/ci.yml`)
`npm run e2e`	Playwright smoke test: the game boots to the menu with WebGPU, and shows the requirements page without it
`npm run assets`	Optimise everything in `art-export/` into `public/assets/` (section 8)
`npm run docs`	Rebuild the design PDFs (`tools/build-docs.sh`)

2.3 Source layout

```
src/
├── main.ts                Boot (section 4)
├── core/
│   ├── Game.ts            Owns engine, scene, loop, top-level state
│   ├── EventBus.ts        Typed publish/subscribe (section 5)
│   ├── Flags.ts           Story flag store
│   ├── AreaLoader.ts      Loads/unloads area asset containers
│   ├── LevelBuilder.ts    Turns glTF extras into game objects (section 7)
│   ├── Time.ts            Game clock, pause, time scale
│   └── Localization.ts    String tables (en, km)
├── player/                Controller, CameraRig, Interactor, Hands
├── systems/               Inventory, Sketchbook, Lamp, Matches, Upkeep,
│                          Hiding, Checkpoints, Saves, Settings, NightCount
├── ai/                    Perception, NoiseSystem, StateMachine, entities/
├── sequences/             Sequencer runtime + shared helper actions
├── audio/                 AudioSystem, Buses, AmbienceZones, Cues
├── render/                RenderSetup, QualityPresets, DeviceProfile, DynamicResolution,
│                          ShadowScheduler, StaticBatcher, VolumetricFog, materials/
├── debug/                 PerfOverlay (F3), Benchmark
├── spikes/                Phase 1 spikes, loaded with ?spike=s1 … (dev builds only)
├── ui/                    diegetic/ (sketchbook pages, gauges), html/ (menus)
├── episodes/ep1/          index.ts, sequences.ts, data/*.json
└── data/                  items.json, notes.json, strings/en.json, strings/km.json
```

2.4 Babylon 9 pitfalls found in Phase 1

Symptom	Cause and fix
"No Physics Engine available"	With tree-shaken imports, `scene.enablePhysics` also needs `Physics/joinedPhysicsEngineComponent`, not only `Physics/v2/physicsEngineComponent`.
`createMultipleRenderTarget is not a function`	WebGPU engine features (multiple and cube render targets, timestamp queries) need `import "@babylonjs/core/Engines/WebGPU/Extensions"`.
Classes that fail `instanceof`; a second load of Babylon	Vite pre-bundles a second copy for code it finds late (lazy spike modules). List every entry in `optimizeDeps.entries`.
Meshes render with nothing	There is no default material when importing by path; give every mesh a material.
Every material recompiles every frame (about 120 MB/s of garbage)	When a shadow caster list goes from empty to filled, Babylon marks every mesh's lights dirty. Filling the list in `renderListPredicate` each frame did exactly that. Keep one caster array per light and update it in place, by index.
A half-resolution post-process runs at full resolution	Setting `postProcess` on a pre-pass effect configuration forces that post-process to full size. Attach the configuration with `_prePassEffectConfiguration` only.
SSAO costs more than its ratio suggests	SSAO2's main pass is always full resolution; only the blur uses the ratio.
Soft point-light shadows look wrong	PCF falls back on cube shadow maps; point lights use Poisson sampling.
`scene.deltaTime` is undefined	Read `engine.getDeltaTime()` instead.

Symptom	Cause and fix
Player wedges on stair edges	The character controller's step-up only moves a few centimetres a frame. Stairs get hidden ramp colliders; `maxStepHeight` is 0 (section 7.1).

03 Modules

Module	Responsibility	Listens / emits
Game	Creates the engine and scene, runs the loop, switches between menu, episode and pause states.	Emits `game:*`
AreaLoader	Loads an area's `.glb` into an asset container, adds or removes it from the scene, disposes it on unload.	Emits `area:loaded/unloaded`
LevelBuilder	Reads node names and extras, creates triggers, interactables, spawn points, patrols, hiding spots, audio zones and lights.	Emits `level:built`
Flags	Key-value store for story state (boolean, number, string). Serialised by Saves.	Emits `flag:changed`
PlayerController	Havok character controller: walk, crouch, crawl, climb stairs; head-bob; footstep timing.	Emits `player:step`, `noise:made`
Interactor	Ray from the screen centre (max 1.8 m); highlights and activates interactables.	Emits `interact:*`
Inventory	Items, counts, combining (key fragments → key).	Emits `inventory:changed`
Sketchbook	Pages and entries unlocked by flags; renders pages to dynamic textures; map and night count. Built in S5: `ui/diegetic/` holds the book model, page painter, layout and drawings; entries are episode data (`episodes/ep1/data/sketchbook.ts`).	Listens to `flag:changed`
Lamp	States open / shuttered / out; oil level and drain rate; flicker; light radius.	Emits `lamp:*`
Upkeep	Generic decaying timers (incense) with warning stages and consequences.	Emits `upkeep:*`
Perception / NoiseSystem	Sight checks and noise events for all AI.	Listens to `noise:made`, `lamp:*`
Entities	Per-entity state machines built from entity data.	Emits `entity:*`, `player:caught`
Sequencer	Runs async scripted sequences; can lock input and the camera.	Emits `sequence:*`
Checkpoints / Saves	Snapshot and restore state; IndexedDB slots; save codes.	Listens to `player:caught`
AudioSystem	Engine creation and unlock, buses, spatial sounds, ambience zones, reverb.	Listens to most events
RenderSetup / PostFx	Engine options, pipeline, quality presets, colour grading per episode.	Listens to `settings:changed`

04 Boot and Game Loop

```
1. main.ts
   ├─ detectDevice(): requestAdapter({ powerPreference: "high-performance" }),
   │    adapter.info (vendor, architecture), userAgentData.mobile, touch points, pointer type
   ├─ classifyDevice() → blocked (no WebGPU, software/fallback adapter, phone or tablet
   │    including iPad) → show requirements page; stop
   │    otherwise → starting preset from the GPU family (Apple → Medium, Intel/AMD
   │    integrated → Low, NVIDIA by architecture …); the player can change it
   ├─ new WebGPUEngine(canvas); await initAsync()          // WebGPU only
   ├─ DynamicResolution: cap render height per preset; lower the scale on sustained
   │    overload, probe upwards now and then (section 11)
   ├─ show HTML loading screen
   ├─ load Havok WASM: await HavokPhysics() → new HavokPlugin(true, havok)
   ├─ scene.enablePhysics(new Vector3(0, -9.81, 0), plugin)
   ├─ load settings + save index from IndexedDB
   ├─ load shared assets (lamp, hands, satchel, sketchbook, UI sounds)
   └─ main menu (HTML) — "Start" click also unlocks audio
2. Start episode
   ├─ AreaLoader.load(first area) → LevelBuilder.build()
   ├─ restore checkpoint or start at beat P01
   └─ engine.runRenderLoop(() => { Time.tick(); systems.update(dt); scene.render(); })
```

Update order each frame	Reason
1. Input → PlayerController	Movement before anything reads the player position
2. Interactor, Lamp, Upkeep	Player-driven state
3. NoiseSystem, Perception, Entities	AI reacts to this frame's player state
4. Sequencer	Scripts can override everything above
5. Audio listener, PostFx parameters	Presentation last

05 Event Catalogue

```
type GameEvents = {
  "flag:changed":      { key: string; value: FlagValue };
  "area:loaded":       { areaId: string };
  "interact:focus":    { id: string } ;  "interact:use": { id: string };
  "inventory:changed": { itemId: string; count: number };
  "lamp:state":        { state: "open" | "shuttered" | "out" };
  "lamp:oil":          { level: number };            // 0..1
  "upkeep:warning":    { id: string; stage: 1 | 2 };
  "upkeep:expired":    { id: string };
  "noise:made":        { pos: Vector3; loudness: number; source: string };
  "entity:state":      { id: string; state: AiState };
  "player:caught":     { by: string };
  "checkpoint:reached":{ id: string };
  "sequence:start":    { id: string };  "sequence:end": { id: string };
  "choice:made":       { id: string; option: string };
  "settings:changed":  { key: string };
};
```

The bus is a small typed wrapper (`on` , `off` , `emit`). Handlers must not throw; errors are caught and logged with the event name.

06 Data Formats

6.1 Items, notes and strings

```
interface ItemDef {
  id: string;                    // "oil_bottle", "key_fragment"
  nameKey: string;               // string-table key
  model: string;                 // "shared/oil_bottle.glb"
  stack: number;                 // max carried, 1 for unique items
  combine?: { with: string; count: number; result: string };
  onUse?: string;                // action id, e.g. "lamp.refill"
}
interface NoteDef {
  id: string;                    // "lore.ep1.L1"
  titleKey: string; bodyKey: string;
  lore: boolean;                 // counts toward Knowledge
  confirmsChant?: string[];      // Episode 5 verse ids it confirms
}
// data/strings/en.json : { "note.ep1.L1.body": "Kanha used to make…", … }
// data/strings/km.json : same keys, Khmer text
```

6.2 Entities

```
interface EntityDef {
  id: string;                    // "ep1_roamer"
  model: string;
  speed: { patrol: number; chase: number };   // m/s
  hearing: { normal: number; crouched: number }; // radius in m
  sight?: { lampRange: number; darkRange: number; fovDeg: number };
  patrol: string;                // Blender patrol path id
  catchRadius: number;
}
```

6.3 Flags

- Keys: `ep<n>.<topic>.<state>` for story, `lore.ep<n>.<id>` for notes, `series.<counter>` for cross-episode values.
- Cross-episode counters: `series.respect` (0–9), `series.vuthy.saved`, `series.vuthy.forgiven`, `series.mother.freed`, `series.chantErrors`.
- Flags are only ever set through `Flags.set()` so every change emits `flag:changed`.

07 Blender and glTF Conventions

7.1 Object name prefixes

Prefix	Becomes	Custom properties (glTF extras)
`trigger_`	Invisible trigger volume (box)	`beat` (e.g. "A2-01"), `once` (bool), `requires` (flag)
`interact_`	Interactable object	`action` ("pickup", "open", "use"), `item`, `note`
`spawn_`	Player or NPC spawn point	`checkpoint` (e.g. "CP3")
`patrol_<id>_NN`	Ordered patrol nodes (empties); numbers may skip	`wait` (seconds); `reach` (bool, test routes: the node's height must be reached)
`hide_`	Hiding spot volume	—
`audiozone_`	Ambience and reverb zone	`ambience`, `reverb`
`light_flame_`	Flickering point light + flame card	`radius`, `intensity`
`col_`	Invisible collision mesh (static physics). Only `col_` meshes collide; render meshes have no physics, so keep colliders simple. A box (8 corners, any rotation) becomes a box collider; any other shape a triangle-mesh collider. Every staircase needs a `col_` ramp from the bottom nosing to the top floor: the character controller can't climb steps (S1).	`surface` ("teak", "dirt", "mud", "leaves", "stone", "water")
`ladder_`	Climbable box; its bounds become the ladder volume. W climbs, looking down and pressing W descends, and at the top the player steps onto the floor. Extend the box about 30 cm above the top floor.	`facing`: direction from the ladder towards the climber in Blender's world axes ("+x", "-x", "+y", "-y")
`nav_`	Navigation mesh source (not rendered)	—

Prefixes are matched at the start of the object name, so visible parts must not use them: name a ladder's mesh `granary_ladder`, not `ladder_granary_mesh`. Collections whose name starts with `_` hold working notes and reference objects and are never exported. Babylon mirrors glTF scenes into its left-handed space (Blender +X becomes –X), so `LevelBuilder` reads positions in world space and builds colliders there.

7.2 Asset naming

- Files: `<scope>_<area>_<name>_<variant>.glb`, e.g. `ep1_house_wall_a.glb`, `shared_lamp.glb`.
- Area scenes: `ep1_area_courtyard.glb`, `ep1_area_house.glb`, `ep1_area_underhouse.glb`, `ep1_area_studio.glb`.
- Materials: `M_<name>`; textures: `T_<name>_<map>` where map is `BC`, `N` or `ORM`.

7.3 Export settings

glTF Binary (`.glb`), +Y up, apply modifiers, export custom properties, everything except `_` collections, no cameras or Blender lights (lights are created from `light_` nodes). `npm run export` runs `tools/export-blend.py` in Blender for every `.blend` in `art-source/blender/` , writing to the same sub-folder of `art-export/` (set `BLENDER` if Blender is not in `/Applications` or on the `PATH`).

Materials: Principled BSDF with image textures for colour (sRGB), normal (Non-Color, through a Normal Map node) and ORM (Non-Color, through Separate Color: green to Roughness, blue to Metallic, red to the Occlusion input of a `glTF Material Output` node group). Turn on Backface Culling for closed meshes; otherwise the material exports double-sided and is drawn from both sides. `tools/s3/make_test_room.py` builds an example scene with every convention.

08 Asset Pipeline

```
# npm run export: art-source/blender/**/*.blend → art-export/**/*.glb (Blender)
# npm run assets: art-export/**/*.glb → public/assets/**/*.glb (tools/build-assets.mjs)
gltf-transform dedup   in.glb  out.glb
gltf-transform prune   out.glb out.glb --keep-leaves true      # keep spawn_ and patrol_ empties
gltf-transform weld    out.glb out.glb
gltf-transform resize  out.glb out.glb --width 2048 --height 2048   # 4096 for shared_ and hero_
gltf-transform etc1s   out.glb out.glb --slots "@(baseColorTexture|emissiveTexture)"
gltf-transform uastc   out.glb out.glb --slots
"@(normalTexture|occlusionTexture|metallicRoughnessTexture)"
gltf-transform meshopt out.glb out.glb
```

KTX2 encoding needs KTX-Software 4.4+ (the `ktx` command; gltf-transform 4 no longer uses `toktx`). Write slot globs as `@(a|b)`: the CLI splits `{a,b}` into separate arguments. Without `--keep-leaves`, `prune` deletes every empty node. Colour maps use ETC1S (small, lower quality); normal and ORM maps use UASTC with Zstandard (larger, no block artefacts in lighting). Spike S3 measured 2.8 MB → 1.6 MB on disk and about 50 MB → 10 MB of GPU memory. UASTC RDO saves about 8% on ORM maps and nothing on normal maps, so it stays off.

Decoders: Babylon fetches the meshopt and KTX2 decoders from its CDN by default. `tools/vendor-decoders.mjs` (run by `npm install`) copies them from `meshoptimizer` and `@babylonjs/ktx2decoder` into `public/vendor/`, bundling the KTX2 decoder into the classic script its worker needs, and `render/decoders.ts` points Babylon at them at boot. Keep both packages at the versions Babylon expects.

09 Scripted Sequences

Sequences are async functions given a context of helper actions. They read like the beat sheet.

```
// episodes/ep1/sequences.ts
export async function A2_01_lampsOut(s: SequenceContext) {
  await s.checkpoint("CP2");
  s.lights.extinguishGroup("tent_hanging", { stagger: 0.15 });
  s.audio.play("gust_cold", { at: "tent_center" });
  await s.wait(1.2);
  s.audio.play("footsteps_wet_above", { at: "house_hall" });
  s.flags.set("ep1.act", 2);
}

export async function A3_07_ritual(s: SequenceContext) {
  await s.checkpoint("CP6");
  s.player.lockMovement();
  await s.waitFor(() => s.inventory.used("bronze_token", "burner_main"));
  await s.waitFor(() => s.inventory.used("sanctified_match", "burner_main"));
  const ok = await s.holdWithoutTurning({ seconds: 25, maxAngleDeg: 100,
                                          during: () => s.presence.approach() });
  if (!ok) return s.fail("CP6");
  s.flags.set("ep1.seal1.broken", true);
}
```

10 Perception and AI

10.1 Perception

- **Hearing:** every `noise:made` event has a loudness (0–1). An entity hears it if distance ≤ its hearing radius × loudness, with walls reducing loudness by 50% (one raycast).
- **Sight:** inside the field of view and within range, then one line-of-sight raycast to the player's head. The range depends on the lamp: open lamp uses `lampRange`; shuttered or out uses `darkRange`.
- Checks run at 10 Hz, not every frame.

10.2 State machine

State	Behaviour	Transitions
Patrol	Follow patrol nodes, waiting where set	Heard noise → Investigate · saw player → Chase
Investigate	Move to the noise position; look around for 4 s	Saw player → Chase · nothing → Return
Chase	Move to the player at chase speed	Within catch radius → Catch · lost sight 5 s → Search
Search	Visit the last known position and two nearby nodes	Saw player → Chase · timeout → Return
Return	Path back to the nearest patrol node	Arrived → Patrol
Catch	Emit `player:caught`; play the catch sequence	Checkpoint restore
Scripted	Controlled entirely by a sequence (walker, presence)	Sequence ends

Pathing uses a navigation mesh baked from `nav_` geometry with Babylon's Recast-based navigation plugin.

11 Rendering

Setting	Low	Medium	High	Ultra
Target	Integrated · Iris Xe, M1, Snapdragon X	1080p · RTX 2060 / M1 Pro	1440p · RTX 3060 Ti	1440p–4K · RTX 3070+
Max render height	900p	1080p	1440p	2160p
Dynamic resolution floor	60%	75%	75%	75%
Lamp shadow map	1024	2048, soft	4096, soft	4096, soft
Live-shadowed flames	0 (baked)	2 at 512	4 at 1024	4 at 1024
Unshadowed lights (clustered)	16	32	64	128
SSAO	Off	Half resolution, 8 samples	Full, 16 samples	Full, 32 samples
Screen-space reflections	Off	Off	Wet surfaces	Wet surfaces
Fog	Exponential height fog	Volumetric, half resolution, 12 steps	Volumetric, half, 24 steps	Volumetric, 75%, 32 steps
Textures	1K	2K	4K	4K
Anti-aliasing	FXAA	TAA	TAA	TAA
Grain, vignette, grading, bloom	On	On	On	On

Low looks plainer but plays the same: darkness, the lamp's shadow and fog are mechanics, so scares are tested on Low too. The values live in `render/QualityPresets.ts`.

- **Device check and presets.** The boot picks a starting preset from the GPU family (section 4); the player can change it in Settings. A short benchmark scene may refine the choice later.
- **Reading floor.** While the sketchbook is open, `DynamicResolution.holdAtLeast(0.85)` keeps the render scale at or above 85% of the preset's height: at Low's 60% floor (540p) page text is unreadable (S5).
- **Dynamic resolution.** The render height is capped per preset and the hardware scaling level set from it. `ResolutionGovernor` lowers the scale in 5% steps when both the current frame and the quarter-second average are over budget for 0.2 s; it ignores single hitches and frames over 250 ms. With a GPU timer it steps up after a second of light load; without one (GPU timestamps read 0 on the reference Mac) frame times are capped at the display refresh, so it probes upwards every few seconds and backs off for longer after a failed probe.
- **Flame shadows are cached.** `ShadowScheduler` renders each flame's cube shadow map once and redraws it only when something moves within its range (at most one per frame on Medium). The lamp's shadow is live every frame.
- **Shadow casters.** Small detail props (offerings, ornaments, carvings) don't cast shadows. Each light keeps one caster array, updated in place (section 2.4).
- **Light range culling.** Each flame light lists only the meshes inside its range (`includedOnlyMeshes`); in S2 this saved about 3 ms a frame.

- **Static batching.** Static meshes are merged by material in 3 m grid cells, so frustum culling still works; S2 started at 11,193 draw calls.
- **Clustered lights.** `ClusteredLightContainer` with its default 64×64 tiles; coarser tiles were slower.
- **Volumetric fog** is a custom WGSL ray-march at half resolution (12 steps on Medium, jittered with interleaved gradient noise and smoothed by TAA), lit by the lamp and up to four flames, then composited over the scene. It reads depth from the pre-pass.
- Post-processing through `DefaultRenderingPipeline` (bloom, ACES tone mapping, grain, chromatic aberration, vignette, colour curves); per-episode colour grading lookup tables.
- Static architecture uses baked lightmaps.
- WebGPU-only features are allowed (compute shaders for volumetrics and particles, snapshot rendering for draw-call reduction), as long as they work in all supported browsers.
- Diegetic pages use `DynamicTexture` or `AdvancedDynamicTexture.CreateForMesh` on the sketchbook mesh.

S2 result on the reference MacBook Pro M1 Pro: Medium at 1080p averages 62.5 fps; Low about 100 fps at 900p. The 1% low (about 30 fps, one slow frame roughly every 1.1 s) is still open; see Development Plan, Phase 1.

Atmosphere (S7)

- **Colour grade.** Each episode keeps its grade as data (`episodes/ep1/data/grade.ts` : saturation, shadow, mid-tone and highlight tints from the palette, black point). `render/grade.ts` bakes it into a 32^3 3D lookup table applied after tone mapping; it replaces the S2 colour curves and costs no more.
- **Eye adaptation.** `render/Atmosphere.ts` sets exposure from the light reaching Sora (her lamp at arm's length = 1, flames by distance). It moves 90% of the way in 2.5 s towards the dark and 0.8 s towards light, and wins back only a quarter of the lost stops, capped at 3× the base: full compensation made rooms brighter with the lamp out. It runs on real elapsed time, so slow frames don't slow it.
- **Night sight.** A faint, cool, unshadowed point light on the camera (range 2.5 m) grows with adaptation and darkness, so shapes within about 2 m show in a dark room (Art Bible darkness rule) while the rest stays black. It counts towards each material's eight lights.
- **Mood and settings.** Fear (0–1) raises the vignette from 1.5 to 4; a scare jolts the aberration from 8 to 32 and fades in about a second; memories raise grain to 14. The grain and aberration settings switch them off; brightness calibration adds ±1 stop.
- **Cost (M1 Pro, Medium, 1080p).** The look together costs about 1–1.5 ms; bloom is the largest part. Effects are switched on and off only when their setting changes, because toggling rebuilds post-processes.

Sketchbook pages (S5)

- **Pages.** A5 pages laid out in millimetres of paper and painted into `DynamicTexture` s with mipmaps and 8× anisotropic filtering: 1024 px high on Low, 1536 on Medium, 2048 on High and Ultra. Four textures: the two open pages and both sides of the turning leaf. Each section starts on a left-hand page; the ribbon tabs are drawn along the bottom of right-hand pages.
- **Text.** Body 8.4 mm of paper (28.6 px at 1080p with the book raised), Khmer 8.8 mm; the large-text setting scales by 1.3. Lines break with `Intl.Segmenter` , which finds Khmer word boundaries (Khmer has no spaces) and never splits a consonant cluster. Fonts load through the `FontFace` API before the first paint.

- **Lighting.** The book is lit by its own reading light from the lamp's side, flickering with the lamp; the room's lamp and flames exclude it. The paper material can also glow faintly (`setPaperGlow`) if playtests find the pages too dark.
- **Drawing order.** The book renders in rendering group 1 with a depth clear, so it never sinks into a wall. A second camera for held items does not work here: the SSAO, fog and TAA pre-pass captures that camera's output and nothing reaches the screen. If held items ever need to skip post-processing, use a utility layer scene instead.
- **Costs (M1 Pro, Medium, 1080p).** Painting a page about 0.5 ms (canvas 0.3 ms, upload and mipmaps 0.9 ms when measured alone); a page turn paints three pages, at most 7.3 ms; the open book adds about 0.4 ms a frame.

12 Audio

Bus	Contents	Notes
Master	Everything	Master volume setting
Ambience	Rain, insects, wind, room tones	Streamed; crossfaded by audio zones
SFX	Footsteps, objects, lamp, matches	Spatial; surface-dependent footsteps
Threat	Entity sounds, scares	Spatial; never ducked
Voice	Dialogue, Sok's recording	Ducks ambience by −6 dB
Music	Stingers and drones	Streamed
UI	Satchel, pages, menus	Non-spatial

- Audio Engine v2 (`CreateAudioEngineAsync`); the engine is unlocked on the first user click (browser autoplay rules).
- Spatial sounds use HRTF panning for threat sounds (positional accuracy matters) and equal-power panning for minor sounds to save CPU. `audio/AudioSystem.ts` creates the buses, attaches the listener to the camera and places sounds by bus (`PANNING`); distance model inverse, full volume within 1 m.
- Panning is tested without listening: the engine also runs on an `OfflineAudioContext` , so S4 renders a footstep from known directions and measures each ear (level, level above 3 kHz, and arrival-time difference). Run this check again after any audio engine update.
- Reverb per audio zone: small wooden room, open courtyard, under-house, large stone hall.
- Formats: Ogg Opus as the primary format, with an AAC fallback for browsers that need it.

13 Saves and Save Codes

```
// IndexedDB key "save.slot1"
{
  "saveVersion": 1,
  "episode": 1,
  "checkpoint": "CP4",
  "clock": "02:00",
  "flags": { "ep1.betel.done": true, "ep1.betel.firstTry": true, "lore.ep1.L1": true, … },
  "inventory": { "oil_bottle": 1, "matches": 7, "sanctified_match": 3, "key_fragment": 1 },
  "lamp": { "oil": 0.64, "state": "open" },
  "playTimeSec": 4230,
  "savedAt": "2026-10-20T14:03:00Z"
}
```

- **Save code** = `"BS1." + base64url(deflate(JSON)) + "." + checksum`. It carries flags and series counters between devices and into the next episode. It is tamper-evident, not secret.
- **Migrations:** `migrations[n]` converts version *n* to *n+1*; old saves are upgraded on load.
- **Checkpoints** store the same structure in memory; reaching a checkpoint also writes slot autosave.

14 Performance and Memory

Rule	Detail
Budgets	As in the Development Plan: about 1,000 draw calls, about 3 million triangles, about 2–3 GB GPU memory per area, first playable area under 150 MB.
Area unload	Dispose the asset container (meshes, materials, textures), stop and dispose area sounds, clear entity references; verify counts in the Inspector return to baseline.
Allocation	No new objects in per-frame code paths (reuse vectors); pool frequently spawned objects.
Profiling routine	Weekly on the reference MacBook Pro M1 Pro with other apps closed and the machine cool: run the S2 benchmark (`?spike=s2&bench=1`) and note average fps, 1% low, draw calls, triangles and JS heap. On a 120 Hz display frame times snap to multiples of 8.3 ms, and browser tabs left open in the background skew results.

15 Testing, Standards and Deployment

Testing

- **Unit tests:** movement maths, head-bob, frame statistics, quality presets, device classification, the resolution governor, the S1 snag detector, level node naming and oriented collider boxes; later Flags, Inventory (including combining), Upkeep timers, Saves (round trip and migrations), save codes, perception maths, AI transitions.
- **Smoke test:** Playwright boots the build in Chrome with WebGPU on and off and checks the menu appears without console errors.
- **Debug tools** (development builds only): jump to any beat or checkpoint, set flags, refill oil, show perception radii, toggle the Babylon Inspector. `debug/saveShot.ts` posts a full-resolution frame to the dev server, which saves it in `debug-shots/` (git-ignored) for checking text and detail at native size.

Code standards

- TypeScript strict mode; no `any` without a comment explaining why.
- One class or module per file; file names match the export (`Lamp.ts`).
- Beat IDs in comments next to the code that implements them (`// A2-06`).
- Small commits with clear messages; a branch per feature.

Deployment

- Static hosting. Serve `.wasm` as `application/wasm` , `.ktx2` as `image/ktx2` , and enable Brotli or gzip compression (it helps code and JSON; optimised `.glb` files are already compressed).
- No run-time CDN: the Havok, meshopt and KTX2 WebAssembly and scripts are served from the game's own origin (`public/vendor/`).
- Hashed file names for long-term caching; `index.html` never cached.
- Check each host's per-file size limit against the largest `.glb` .